

—— // REWRITING THE DESKTOP // ——

The Operator

First Edition

THE COMPANION

the port · the core · the shell



MENKE
TECHNOLOGIES

Copyright and Attribution

The Operator — building a stack with coding agents

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by Anthropic, OpenAI, Google, Microsoft, GitHub, Cursor, the Rust Foundation, the Zsh project, the tmux project, the GNU Project, or any other product or project named in these pages.

The software this book describes

The systems in this book span both licensing families in the MenkeTechnologies catalog. The free and open-source works — among them `zpw`, `zsh-more-completions`, `zsh-expand`, and the zsh plugin fleet — ship under the terms in their own repositories. The proprietary works — among them `zshrs`, `strykelang`, `fusevm`, `zmax`, `zvcs`, and the desktop application fleet — ship under the terms in their own repositories' `LICENSE` files, which state that the software is “proprietary and confidential”, is distributed under the SPDX identifier `UNLICENSED`, and grants no license to use, copy, modify, or redistribute it without prior written permission.

Reading this book conveys no license to the software it describes.

The tools this book describes

This book names the coding agents and models it was written alongside: Claude and Claude Code from Anthropic, Cursor, GitHub Copilot, OpenAI's models, Google Gemini, and locally served models through Ollama. None of those products' source is used, derived from, or redistributed here. They are named because the book is an account of working with them, and they remain the property of their own owners.

Where this book reports what an agent did, it reports what was observed in a working session against a specific repository at a specific commit. Where it reports a number, that number was derived from the repositories by a command recorded in the text. Where it makes a claim it could not measure, it says so.



MENKE
TECHNOLOGIES



MT PUBLISHING



MT APP

Contents

Copyright and Attribution	1
Before You Begin	3
Chapter 1 — The Measurement	6
Chapter 3 — What the Numbers Do Not Say	10
Chapter 5 — Parallel Sessions	14
Chapter 6 — Decomposition	16
Chapter 8 — Context Failure	18
Chapter 9 — Convergent Reinvention	19
Chapter 10 — Confident Wrongness	21
Chapter 12 — Tool-Grounded Claims	24
Chapter 13 — Testing Under Volume	26
Chapter 15 — Case Study: fusevm	29
Chapter 16 — Case Study: zshrs	30
Chapter 17 — Case Study: strykelang	31
Chapter 19 — Case Study: zmax and zvcs	33
Chapter 20 — The Foundation That Predates the Method	35
Chapter 22 — What Expertise Is For Now	38
Chapter 23 — Economics of a One-Person Stack	40

Before You Begin

Provenance. Every figure in this book was derived on 2026-08-12 from the working checkouts under the MenkeTechnologies meta repository. Where a number in this book cannot be reproduced by one of the commands below, the text says where it came from instead.

The three commands behind every table in this book:

```
# Tracked Rust lines in a repository - excludes target/ and untracked files
git ls-files '*.rs' | xargs wc -l | tail -1

# Commit count
git rev-list --count HEAD

# First and last commit dates
git log --reverse --format=%cd --date=short | head -1
git log -1 --format=%cd --date=short
```

This is a book about a working method, written by the person who used it, about the stack he used it on.

Between April and August of 2026, a single author produced roughly 5.75 million lines of tracked Rust across 116 repositories: a bytecode virtual machine with a JIT compiler, a compiled Unix shell, a programming language, a terminal multiplexer, an editor, a version-control system, and a fleet of desktop applications. The same author had spent the previous nine years building a shell environment in zsh — 6,332 commits between 2017 and 2026 — at a rate that looks, in retrospect, like a different profession.

The difference between those two periods is the subject of this book.

It is tempting to describe that difference as a tool acquisition, the way one might describe adopting a new editor or a faster compiler. That description is wrong in a way that matters. Coding agents did not make the existing work faster. They changed which work is worth doing, where the bottleneck sits, what a defect looks like, and what a senior engineer is actually for. A person who adopts them and keeps working the same way gets a modest speedup and a lot of code they have not read. A person who reorganizes around them gets something else.

This book is an account of the reorganization.

What this book is not

It is not an argument that agents write good code. They frequently do not. A substantial portion of this book catalogs the specific ways they fail, because the failures are more instructive than the successes and because anyone who works this way for long enough develops the same catalog independently.

It is not a claim that the code described here is uniformly excellent. Five million lines produced in four months contains generated code, vendored interfaces, test fixtures, and repetition. Chapter 3 is devoted entirely to what the line counts do and do not mean, because a book that opens with a large number and never interrogates it is marketing rather than engineering.

It is not a tutorial. There are no prompt templates here. Prompt templates are the least durable part of this practice and the part most likely to be obsolete by the time a book reaches a reader. What survives is the structure around the prompts: how work is decomposed, what context is supplied, how output is verified, and which categories of decision are never delegated.

And it is not neutral about the tools. The book names Claude Code, Cursor, Copilot, Gemini, and locally served models, and it says plainly which ones were used for what and where each fell short. Vendor-neutrality in a book about tools is usually a way of avoiding the specific claims that would make the book useful.

What this book is

It is a description of one method, with the artifacts that method produced available for inspection. The reader who wants to check a claim can clone a repository and run the command in the provenance note.

The method has five components, each of which gets a part of this book:

The context file. The single highest-leverage artifact in agentic development is not the prompt; it is the standing document that tells an agent what the project's conventions are, which patterns are forbidden, what already exists that must be reused, and what verification is required before work is considered done. Most agent failure is context failure. Part II is about writing that document and about what happens when it is absent.

Parallel decomposition. A single agent session working through a task list sequentially wastes most of the available capacity. The method described here runs many sessions concurrently against separate areas of a codebase, which introduces its own class of problem — convergent reinvention, conflicting abstractions, integration debt — that the context file exists partly to prevent.

The failure taxonomy. Agents fail in patterned, recognizable ways. Confident wrong output in unfamiliar territory. Silent reinvention of an abstraction that already exists three directories away. Tests written to pass rather than to catch. And the most dangerous failure of all: modifying the instrument that measures correctness instead of fixing the code the instrument is measuring. Part III is the catalog.

Verification discipline. Nothing is accepted on plausibility. This is the rule that costs the most and matters the most, and it is the one that separates a working practice from an accumulation of unread code. Part IV is about what verification actually consists of when the volume of change exceeds what any human can read line by line.

Judgment retention. There is a set of decisions that never goes to an agent: architecture, boundaries, what belongs in the system at all, and anything that touches the measurement of correctness. Part VI is about where that line sits and why it moves more slowly than the tooling does.

How to read this book

The book runs in six parts.

Part I — The Measurement establishes what actually happened, in numbers, and then spends a chapter attacking those numbers. Chapter 1 is the before-and-after. Chapter 2 is the stack under discussion. Chapter 3 is the honest accounting of what 5.75 million lines does and does not represent.

Part II — Method is the working practice: the context file (Chapter 4), parallel sessions (Chapter 5), task decomposition (Chapter 6), and the review loop (Chapter 7).

Part III — Failure is the taxonomy: context failure (Chapter 8), convergent reinvention (Chapter 9), confident wrongness (Chapter 10), and the audit-tool problem (Chapter 11), which is severe enough to warrant its own chapter and its own permanent rule.

Part IV — Verification is what replaces reading every line: tool-grounded claims (Chapter 12), test strategy under volume (Chapter 13), and the specific question of how you know a large diff is correct (Chapter 14).

Part V — Case Studies walks the stack: fusevm (Chapter 15), zshrs (Chapter 16), strykelang (Chapter 17), ztmux (Chapter 18), zmax and zvecs (Chapter 19), and the free and open-source works that predate the method (Chapter 20).

Part VI — What Does Not Change is the closing argument: the reviewer bottleneck (Chapter 21), what expertise is now for (Chapter 22), and the economics of a one-person stack (Chapter 23).

A reader interested only in the practice can read Parts II through IV and skip the case studies. A reader skeptical of the whole enterprise should read Chapter 3 and Chapter 11 first, since those are the two chapters where the book argues against itself.

Chapter 1 – The Measurement

The clearest evidence available for what changed is a single author working in a single domain across two periods, because it holds constant nearly everything that usually confounds this comparison: talent, taste, domain knowledge, tolerance for tedium, and the willingness to work long hours.

The baseline

`zpw` is a zsh environment: 506 subcommands, more than two thousand aliases, and a completion corpus that grew alongside it. Its first commit lands on 2017-05-31. Its most recent, at the time of writing, lands on 2026-07-11. Between those dates it accumulates **6,332 commits** — a little over nine years of sustained, unbroken work by one person on one system, in a language that offers no compiler, no type system, and very little tooling.

That is the baseline. Not a bad one: 6,332 commits over nine years is a consistent output that most side projects never approach, and the result is a functioning environment that other people install and use.

The comparison

`zshrs` is the Rust rewrite of zsh. Its first commit lands on 2026-04-24. Its most recent lands on 2026-08-06. Between those dates it accumulates **6,431 commits** across **824 tracked Rust files** totaling **854,674 lines**.

Nine years of zsh work produced 6,332 commits. Three and a half months of Rust work produced 6,431.

The commit counts are close enough to be a coincidence of measurement, and comparing commits across languages and eras is a weak metric in isolation. The line counts are the harder number: 854,674 lines of tracked Rust in 104 days is approximately **8,200 lines per day, every day, including the days nothing was committed**.

No one writes 8,200 lines of Rust per day by hand. That number is not a claim about typing speed; it is a claim about a different production process, and the rest of this book is about what that process is.

The stack in aggregate

`zshrs` is not the whole of it. Measured on 2026-08-12 across the meta repository's submodules:

Repository	Rust files	Tracked lines	Commits	First → last commit
<code>zshrs</code>	824	854,674	6,431	2026-04-24 → 2026-08-06
<code>zvcs</code>	1,928	616,918	440	2026-07-19 → 2026-08-12
<code>strykelang</code>	660	497,522	1,863	2026-04-08 → 2026-08-04
<code>zmax</code>	564	436,097	1,454	2026-06-24 → 2026-08-12

Repository	Rust files	Tracked lines	Commits	First → last commit
<code>ztmux</code>	308	169,446	239	2026-07-02 → 2026-08-10
<code>fusevm</code>	81	155,051	194	2026-04-24 → 2026-07-25
<code>zoffice-core</code>	22	79,599	704	—
<code>zpdf-core</code>	30	30,575	273	—

Across the full meta repository: **197 submodules**, of which **116 contain tracked Rust**, totaling **5,754,957 lines**, against **109,249 commits** in aggregate across all submodules including the pre-2026 shell work.

`zvcs` is the extreme case and deserves its own note. 616,918 tracked lines across 1,928 files, with a first commit on 2026-07-19 and a most recent on 2026-08-12 — twenty-four days. That is an arithmetic rate above 25,000 lines per day, which should immediately provoke the question Chapter 3 is written to answer: what is actually in those files?

What the comparison does not prove

It does not prove the newer code is better. It is not the same language, not the same problem, and not the same design constraints; a shell environment in `zsh` and a shell implementation in Rust differ in almost every dimension that matters to line count. Rust is more verbose than `zsh` per unit of behavior, and a from-scratch implementation carries structural code — error types, trait definitions, module scaffolding — that an environment layered on top of an existing shell never needs.

It does not prove the method generalizes. One author, one stack, one four-month window, no control group.

What it does establish is that something changed by roughly an order of magnitude in throughput, for the same person, in adjacent domains, across a boundary that coincides exactly with the adoption of coding agents as the default way of working. That is not proof. It is the observation the rest of the book tries to explain, and Chapter 3 is where the observation gets its adversarial review. # Chapter 2 — The Stack Under Discussion

Before the method, the artifacts. This chapter is a short tour of what was built, because the case studies in Part V assume the reader knows what each system is for.

fusevm — the shared machine

`fusevm` is a bytecode virtual machine with a Cranelift JIT backend: 81 tracked Rust files, 155,051 lines, 194 commits between 2026-04-24 and 2026-07-25.

It is the smallest repository by file count in the top tier and the most structurally important, because everything else in the language family compiles down to it. A bytecode format, a register allocator, a JIT compilation path through Cranelift for x86-64 and aarch64, and an ahead-of-time native binary path for the mature frontends.

The file-to-line ratio here is worth noting against the others: 81 files holding 155,051 lines is an average near 1,900 lines per file, which is what a VM looks like — large dispatch loops, large instruction-definition tables, and a small number of very dense modules.

zshrs — the compiled shell

`zshrs` is a Rust implementation of `zsh`: 824 files, 854,674 lines, 6,431 commits in 104 days.

The design premise is stated as a rejection of the alternative. Zsh’s ecosystem has accumulated a stack of startup optimizations — `zinit` turbo mode, powerlevel10k’s instant prompt, `zcompile`, `.zcompdump` caching, lazy autoload — each of which exists to work around the same underlying constraint: an interpreter that reparses its configuration on every launch. The premise of `zshrs` is that when a community accumulates N+1 well-known workarounds for one problem, the problem is not fixable in userspace.

So: bytecode compilation with an mmap’d `rkvy` cache, a JIT path, a persistent worker pool, in-process fork-free coreutils, and a hard rule against shipping any startup trick of its own. The acceptance criterion is “first paint equals full functionality” rather than “first paint equals a visible prompt while the real work happens behind it.”

Whether it meets that bar is a question for its own reference manual. What matters here is that the ambition — a compiled shell with drop-in zsh compatibility — is the kind of project that a single author would not previously have attempted, because zsh’s own C implementation is roughly 130,000 lines and represents the compatibility floor rather than the ceiling.

strykelang – the language

`strykelang` is a programming language: 660 files, 497,522 lines, 1,863 commits between 2026-04-08 and 2026-08-04. It is the oldest repository in the 2026 cohort by first-commit date, which makes it the first system built with the method.

It is a Perl-5 superset with a `--compat` mode, running on `fusevm`, with a work-stealing parallel runtime via Rayon. Its syntactic lineage is explicitly polyglot: threading macros from Clojure, arrow forms from Racket, underscore placeholders from Scala, sigils and topic variables from Perl, and string-interpolation and inspection conventions from Ruby.

The design decision most relevant to this book is that `strykelang` treats LLM invocation as a language primitive. An `ai` builtin in the standard library issues schema-structured model calls with tool and function calling and Model Context Protocol support, with no SDK and no import. The language that was built with agents also exposes agents as a first-class construct.

ztmux – the multiplexer

`ztmux` is a terminal multiplexer: 308 files, 169,446 lines, 239 commits between 2026-07-02 and 2026-08-10.

It is the cleanest test case in the stack for a specific claim, because it was built against an existing reference implementation with a mechanical definition of correctness. Chapter 18 covers it in detail, including the byte-parity test corpus, because “does the port produce identical output to the original” is the rare specification that admits an unambiguous answer.

zmax – the editor

`zmax` is a Rust Emacs port with a Vim-modal core and Spacemacs-style layers: 564 files, 436,097 lines, 1,454 commits between 2026-06-24 and 2026-08-12.

Editors are among the least tractable software to port, because the surface area is enormous and the expectations are exacting: every keybinding, every mode, every buffer-local variable, and a Lisp dialect underneath it all. `zmax` embeds ten language frontends in its core, which is where it stops being an editor port and becomes a demonstration of the shared-VM thesis.

zvcs – version control

`zvcs` is a version-control system: 1,928 files, 616,918 lines, 440 commits between 2026-07-19 and 2026-08-12.

Twenty-four days. This is the repository that most sharply raises the questions Chapter 3 addresses, and the honest answer involves the file count as much as the line count: 1,928 files at an average of 320 lines each is a very different shape from `fusevm`'s 81 files at 1,900 lines each, and shape is diagnostic.

The desktop fleet

Beyond the language and terminal work sits a fleet of Tauri v2 desktop applications over pure-Rust `*-core` engines with C ABI surfaces: a PDF editor, a raster image editor, an office suite, an email client, an API client, a reference manager, a container GUI, and others. `zoffice-core` holds 79,599 lines across 22 files; `zpdf-core` holds 30,575 across 30.

These are covered by their own book and are relevant here only as evidence of breadth: the method was not applied to one problem shape but to VMs, shells, languages, editors, version control, and GUI applications in the same window.

The free and open-source foundation

Predating all of it, and continuing alongside, is the FOSS work: `zpw` itself, `zsh-more-completions` — a completion corpus in the tens of thousands of files — `zsh-expand`, `zsh-cargo-completion`, and roughly twenty published zsh plugins, plus contributions to zsh, Homebrew, zinit, and Oh My Zsh. The Homebrew zsh completion in particular ships to every Homebrew user on macOS.

That body of work was built without agents, over nine years, and it is the reason the 2026 cohort was possible: the method described in this book requires an operator who already knows what correct looks like in the target domain. Chapter 20 returns to this point, because it is the single largest caveat on everything else here.

Chapter 3 – What the Numbers Do Not Say

A book that opens with 5.75 million lines and does not immediately attack that figure is not an engineering document. This chapter is the attack.

Line counts are a measure of volume, not value

`wc -l` counts newlines. It does not distinguish a dense dispatch loop from a generated match arm, a hand-designed trait hierarchy from a vendored constant table, or a subtle concurrency fix from a thousand lines of test fixtures. Every line count in Chapter 1 is a volume measurement, and volume is the least interesting property of a codebase.

The reason to report it anyway is that it is reproducible. Anyone with the repositories can run the command and get the same answer, which is more than can be said for “productivity,” “velocity,” or any of the constructed metrics that usually appear in this argument. A weak metric that can be independently verified is preferable to a strong metric that cannot.

What is actually in those lines

Four categories inflate any large Rust line count, and all four are present here.

Generated code. A language implementation with seventeen frontends generates a great deal of itself: lexer tables, parser state machines, AST node definitions, visitor scaffolding, and the lowering paths from each frontend onto shared bytecode. Much of that is emitted from grammar definitions rather than typed by anyone. It is real code that must compile and must be correct, but attributing it to authorship of any kind — human or agent — misstates what happened.

Tests and fixtures. `ztmux`’s byte-parity corpus is 1,080 cases. Each case is input, expected output, and harness scaffolding. Test data is the correct thing to have and the wrong thing to count as implementation.

Interface surface. Rust is verbose at boundaries. A `*-core` crate exposing both a native Rust API and a C ABI writes every type twice, plus the `unsafe` marshalling between them, plus the error conversions. That is necessary work and it is not proportional to the behavior it enables.

Repetition across frontends. Seventeen language frontends onto one VM share an architecture, which means seventeen lexers that differ in their token tables and resemble each other everywhere else. The second frontend costs a fraction of the first; the tenth costs a fraction of the second. The line count does not reflect that decay.

The file-shape diagnostic

Comparing the two extremes in the table makes the point:

- `fusevm`: 81 files, 155,051 lines — **1,914 lines per file**
- `zvcs`: 1,928 files, 616,918 lines — **320 lines per file**

A VM concentrates complexity into a few very large modules. Its line count is mostly irreducible: instruction definitions, dispatch, register allocation, JIT lowering. A 616,918-line version-control system spread across 1,928 files at 320 lines each has a different composition — many small modules, high structural regularity, and a much larger proportion of interface, test, and generated material.

Neither shape is better. But a reader estimating “how much thinking is in this repository” should weight them very differently, and the aggregate 5.75 million figure hides the distinction entirely.

The rate figures are worse

Chapter 1 reported roughly 8,200 lines per day for `zshrs` and above 25,000 for `zvcs`. Both figures are arithmetic artifacts, computed by dividing final line count by the span between first and last commit. They assume work was uniformly distributed across that span, which it was not, and they include every line that was ever committed rather than the net of additions and deletions.

A more honest characterization: these repositories reached their current size in a number of weeks that would have been implausible for one person in 2024. The precise rate is not meaningful. The order of magnitude is.

What would be a better measurement

Several things, none of which are available:

Net lines after churn. Additions minus deletions across history, which would discount the substantial rewriting that agentic development produces.

Defect density. Bugs per thousand lines, measured against a comparable hand-written codebase. There is no comparable codebase and no bug database.

Behavioral coverage. For the ports specifically — `ztmux`, `zmax`, `zshrs` — the fraction of the original's behavior reproduced. `ztmux`'s parity corpus is the only place in the stack where this is measured, which is precisely why Chapter 18 leans on it so heavily.

Time to comprehension. How long it takes a competent engineer who did not write the code to understand a given module well enough to change it safely. This is the metric that matters most for maintenance and the one nobody knows how to measure.

The honest summary

A single author produced an unusual volume of Rust in an unusual amount of time. The volume includes generated code, test data, and interface repetition in proportions that are not separately measured. The systems compile, and the one system with a mechanical correctness criterion passes it. Beyond that, claims about quality in this book are qualitative and should be read as such.

That is the caveat. The rest of the book is about the method that produced the volume, and the method is more defensible than the number. # Chapter 4 — The Context File

The highest-leverage artifact in agentic development is not the prompt. It is the standing document that an agent reads before it does anything.

Prompts are ephemeral. They describe one task, they are written under time pressure, and they are gone when the session ends. The context file persists across every session, every task, and every agent, and it encodes the things that would otherwise have to be re-explained — or, more commonly, would not be explained at all, and would be violated silently.

What goes in it

Six categories, in rough order of value.

Forbidden patterns. The most valuable content is negative. An agent generating plausible code will reach for the idiomatic solution, and in a mature codebase the idiomatic solution is frequently wrong for local reasons. A rule of this shape:

```
NEVER use `sed`. Use `perl` for all stream editing and in-place edits.
BSD sed needs -i '' (empty backup arg); GNU sed needs bare -i. The two
forms fail silently on the other flavor. Homebrew's gnu-sed shadows
/usr/bin/sed on macOS, so `sed -i ''` breaks even there: GNU parses ''
as an empty script, the real expression becomes a filename, rc=2, and
the failure is usually swallowed by `|| true`.
```

```
Correct: perl -i -pe 's/.../.../' file
```

exists because a version-stamping step once silently no-opped and shipped a release with a stale version string. An agent has no way to know that. Written down once, it stops recurring forever.

Existing abstractions that must be reused. The single most common structural failure in agentic development is convergent reinvention: an agent writes a perfectly good utility that already exists three directories away, because it did not know to look. Chapter 9 is devoted to this. The mitigation is a section of the context file that names the shared utilities and says explicitly that new instances must route through them.

Verification requirements. What must be true before work is reported as done. Tests run and passing, with the output cited. Files read rather than assumed. Benchmarks executed rather than estimated. Stating this in the context file moves it from something the operator has to demand every time into something the agent does by default.

Project conventions. Naming, module layout, error-handling style, comment density, how the surrounding code reads. This is the category most people write first and it is the least important of the six, because style violations are visible in review and the others are not.

Domain constraints. The things that are true about this system that are not true generally. That a shell must not print anything at startup. That a particular data structure is memory-mapped and therefore cannot contain pointers. That a function is on a hot path measured in nanoseconds.

The measurement prohibition. Tests, lint configurations, coverage tooling, and report generators are off-limits. This is important enough that Chapter 11 is about nothing else.

What the absence of one looks like

A codebase built by agents without a context file exhibits a recognizable pathology: fourteen implementations of the same filter, each slightly different, each individually reasonable. No single review catches it, because each implementation is fine on its own terms. The problem is only visible in aggregate, and by the time it is visible it is expensive.

The same pathology appears in error handling — three different error types for the same failure mode — and in configuration, and in logging. Anywhere a decision could reasonably be made more than one way, an agent without standing context will make it differently each time, and the differences will not be caught by any individual code review.

Why it works

The mechanism is unglamorous: agents fail when they lack information, and the context file is information they cannot otherwise obtain. An agent can read the code it is pointed at. It cannot read the code it was not pointed

at, cannot know which of several reasonable approaches this project settled on two months ago, and cannot know which mistakes have already been made and corrected.

Everything in the context file is something the operator knows and the agent does not. Writing it down is the entire trick.

The maintenance problem

Context files rot. A rule written in April describes a codebase that has since changed; a forbidden pattern gets fixed upstream and the prohibition becomes noise; a named utility gets renamed. A stale context file is worse than none, because it produces confident wrong behavior rather than uncertainty.

The practice that works: when a rule is violated in a way that survives review, the rule was insufficiently clear, and the fix is to rewrite the rule rather than to correct the instance. When a rule stops applying, delete it. Treat the file as code, with the same standard for accuracy.

A note on publication

Context files are increasingly something employers ask to see, on the reasonable theory that a candidate's context file reveals more about how they work than any description of their process. That makes them a public artifact, which changes what belongs in them: a context file that is going to be read by strangers should contain project rules, not personal notes.

Chapter 5 – Parallel Sessions

A single agent session working sequentially through a task list uses a small fraction of the available capacity. The constraint is not model throughput; it is that one session touches one area at a time while the operator waits.

The method described here runs many sessions concurrently — in practice, a grid of terminal panes, each with an agent working a scoped task against a different part of the codebase, with the operator moving between them.

What parallelism buys

The obvious gain is wall-clock. Sixteen sessions working sixteen tasks finish substantially faster than one session working sixteen tasks, and the speedup is close to linear as long as the tasks are genuinely independent.

The less obvious gain is that it changes what work is worth starting. When a speculative refactor costs a session that is running anyway alongside fifteen others, the calculus for “is this worth trying” shifts. Experiments that would never justify a day of hand-coding become cheap enough to run and discard.

What parallelism costs

Integration debt. Sixteen sessions produce sixteen diffs that were each written against a codebase state that no longer exists by the time they land. Most of the time this is harmless. Occasionally two sessions modify the same interface in incompatible ways, and the cost of discovering that at integration time is higher than the cost of the original work.

Convergent reinvention at scale. The reinvention problem from Chapter 4 gets worse in proportion to concurrency. Two agents that both need a path-normalization helper, working simultaneously, will both write one. Neither is wrong; the result is two helpers.

Review saturation. This is the real limit. Sixteen sessions produce output faster than one person can review it, and the temptation to accept work on plausibility scales directly with how far behind the review queue falls. Part IV is about managing this, but the honest statement of the constraint is that parallelism is bounded by review capacity, not by tooling.

Scoping tasks for concurrency

The properties that make a task safe to run in parallel:

Bounded blast radius. The task touches one module, or one layer, or one frontend. Tasks that cross architectural boundaries should run alone.

Independent verification. The task’s correctness can be established without reference to the other in-flight work — usually meaning it has its own tests.

A stated interface. If the task needs to call something another session is building, the interface is specified in advance by the operator rather than negotiated between sessions after the fact.

No shared-state mutation. Two sessions editing the same file is the common failure. Two sessions editing the same *concept* — both adding cases to one dispatch table, say — is the subtle one.

The operator's role under concurrency

With one session, the operator is a collaborator. With sixteen, the operator is a dispatcher: assigning scoped work, specifying interfaces before they are needed, watching for the failure signatures in Part III, and maintaining the review queue.

This is a different job from programming, and it is worth being explicit that some people will not enjoy it. The work becomes specification, review, and integration, with the pleasure of writing code substantially removed. What replaces it is throughput of a kind that is not otherwise available to one person.

Chapter 6 – Decomposition

The unit of agentic work is the scoped task, and scoping is the skill that determines whether the method produces a system or a mess.

Diff size is the governing constraint

A 2,000-line diff is unreviewable regardless of who or what wrote it. The reviewer skims, approves, and the code enters the system unread. This is true of human pull requests and it is more true of agent output, because there is no author to ask.

Everything about decomposition follows from keeping diffs inside the size a person will actually read. In practice that is a few hundred lines, and the discipline is to split work until it fits rather than to review faster.

Vertical over horizontal

Two ways to split a feature:

Horizontally, by layer: one task for the data model, one for the service, one for the API surface, one for tests. Each task is small and none is independently verifiable, because nothing works until all four land.

Vertically, by behavior: one task that implements one complete behavior through every layer, with its tests. The diff is larger but it is verifiable on its own, and it either works or it does not.

Vertical splitting is almost always correct for agentic work, because the verification property matters more than the size property. A horizontal split produces four diffs that must be integrated before anything can be tested, which is precisely the situation where unverified code accumulates.

Specifying the interface first

When two pieces of work must meet, the operator writes the interface — the trait, the function signature, the message schema — before either session starts. This is the single highest-value intervention in the whole method, because it is the one thing agents are worst at negotiating among themselves and the one thing a human with the whole system in mind is best at.

An agent given a specified interface implements against it. Two agents left to discover an interface between them produce two incompatible guesses.

What never gets decomposed to an agent

Architecture. Whether a subsystem should exist. Where a boundary goes. What the error model is. Which dependency to take on.

These are not withheld because agents produce bad answers — they often produce reasonable ones. They are withheld because these decisions are expensive to reverse and their consequences appear months later, which is exactly the horizon over which an agent has no stake and no memory. The operator holds them because the operator is the one who will still be there. # Chapter 7 — The Review Loop

Review under agentic development is a different activity from review of human work, because the questions worth asking are different.

Reviewing a colleague's pull request, one assumes competence and intent, and looks for the mistakes a competent person makes: an unhandled edge case, a race, a misunderstanding of the requirement. Reviewing agent output, competence is not a safe assumption in unfamiliar territory and intent does not exist at all. The questions become:

Does this belong here? An agent asked to add a feature will add it wherever it was pointed, without an opinion about whether that is the right module. Placement is the first thing to check and the thing agents get wrong most consistently.

Does this already exist? Before reading the implementation, search the codebase for the thing it implements. Chapter 9 is about how often the answer is yes.

What does it actually do? Not what the comment says, not what the function name implies. Agent-written comments describe intent, and intent and behavior diverge silently.

How do I know it works? Not “does it look right.” What was run, and what was the output.

Reading order

The productive order inverts normal review. Start with the tests, because tests describe claimed behavior and are quick to assess for whether they would catch anything. Then the interfaces — signatures, types, error paths — because those are where placement and design errors live. Then the implementation, which by that point can be read for whether it matches what the first two promised.

A large fraction of defects are visible in the first two passes without reading the implementation at all.

The plausibility trap

The dangerous property of agent output is that it is uniformly plausible. Human code carries signals — hesitation, inconsistent naming, a comment saying “not sure about this” — that mark where the author was uncertain. Agent output has no such signal. The section the model was confident about and the section it fabricated look identical.

The only defense is to stop using plausibility as evidence. Not “does this look correct” but “what did I run to establish that it is.” That rule is expensive and it is the difference between a codebase and a pile.

When to discard rather than fix

Agent output that is substantially wrong is usually cheaper to regenerate than to repair, because the cost of a fresh attempt is minutes and the cost of understanding someone else's wrong approach well enough to correct it is not.

The exception is when the output is wrong in an *interesting* way — when it reveals that the task was under-specified, or that an assumption in the surrounding code is false. Then the wrongness is information and the right move is to fix the specification before regenerating.

Chapter 8 – Context Failure

Most agent failure is context failure: the model lacked information it could not obtain, produced a reasonable answer given what it had, and the answer was wrong for reasons invisible from inside the session.

This is the most common failure category and the most fixable, which is why it comes first in the taxonomy.

The shapes it takes

Reaching for the general solution in a specific codebase. The idiomatic approach is usually right, which is what makes it dangerous when it is not. A codebase that avoids a standard library facility for a measured reason will get that facility reintroduced by any agent that was not told.

Solving the stated problem rather than the real one. A request to “make this faster” produces micro-optimization when the actual cost is an N+1 query one layer up. The agent optimized what it was shown.

Missing the constraint that lives elsewhere. A function is called from a hot path, or inside a signal handler, or from FFI where panics cannot cross the boundary. None of that is visible at the definition site.

Assuming a convention that does not hold. That errors propagate with `?`, that logging goes to a particular facility, that a type is cheap to clone.

Why more context is not automatically the fix

The naive response is to supply everything: the whole codebase, every convention, all history. This degrades results. Long contexts dilute attention, and a rule buried on page forty of a context file is functionally absent.

What works is *relevance*, not volume. The rules that apply to this task, the abstractions that exist in this area, the constraints that bind this code path. A short, accurate, task-relevant context outperforms a long comprehensive one, which is why the context file needs pruning as much as it needs writing.

The operator's tell

There is a recognizable moment in a session when the operator realizes the agent is working from a wrong model — the code is coherent but answers a question nobody asked. The correct response is to stop and repair the context rather than to correct the output, because the wrong model will produce more wrong output for as long as it persists.

The instinct to just fix the code and move on is strong, and it is how context failures become recurring costs.

Chapter 9 – Convergent Reinvention

An agent asked to solve a problem solves it. It does not first survey the codebase to determine whether the problem has already been solved. Multiply by many sessions and the result is a codebase with several implementations of everything.

The pathology

The symptoms, in rough order of appearance:

- Two utility functions with different names doing the same normalization
- Three error types representing the same failure
- Four slightly different retry loops
- Fourteen filter implementations, each individually defensible

The last is not hyperbole; it is the shape the problem takes when it goes unaddressed through a period of high-concurrency work. Each implementation passed review because each was reasonable in isolation. The defect is only visible in aggregate.

Why review does not catch it

Code review examines a diff. The reinvention is not in the diff — it is in the relationship between the diff and code elsewhere in the repository. A reviewer who does not happen to remember the existing implementation approves a perfectly good function.

Human teams have partial protection: a reviewer who wrote the original utility recognizes it. That protection scales poorly and vanishes entirely when the volume of change exceeds what any one person holds in memory.

What works

Naming the abstractions in the context file. A section listing the shared utilities, what they do, and an instruction that new instances route through them. This catches the common cases.

Searching before implementing. Making it part of the task specification: before writing a helper, grep for one. Agents comply with this reliably when told, and never do it unprompted.

Periodic aggregate review. Not reviewing diffs but reviewing the codebase: searching for duplicated concepts, comparing similar function names, looking for the same string literal in several places. This is the only reliable detector and it is a distinct activity that has to be scheduled.

Building the shared thing first. When a pattern will recur — and the operator usually knows in advance — write the shared implementation before the first use site, then point every subsequent task at it. This is cheap in advance and expensive to retrofit.

The deeper problem

Convergent reinvention is a symptom of a system with no memory. Each session starts fresh, and the only durable memory is what the operator has written down or what is discoverable in the code itself. The context file is a memory prosthesis, and its quality determines how much the system forgets between sessions.

Chapter 10 – Confident Wrongness

The failure mode with no warning signal: output that is fluent, well-structured, correctly formatted, and wrong.

Where it concentrates

Unfamiliar APIs. Agents produce plausible calls against libraries they do not know well — right shape, wrong method name, or a real method with different semantics than assumed. The code often compiles, which removes the cheapest detector.

Version boundaries. An API changed between major versions and the training data contains both. The output blends them.

Rarely-exercised paths. Error handling, cleanup, cancellation, and the interaction of features that are individually well-covered. Anything that is correct in the common case and wrong at the margin.

Numerical and temporal reasoning. Off-by-one in date arithmetic, timezone handling, integer overflow at boundaries. The code reads correctly and computes incorrectly.

Why experience is the defense

Reviewing generated code in a stack you know cold is a different activity from reviewing it in a stack you are learning. In a familiar stack, wrongness produces a feeling of friction — this is not how that API behaves, that error type does not exist, nobody writes it that way. That friction is the detector, and it is built from years of writing the code by hand.

In an unfamiliar stack there is no friction, and the review degrades to checking that the code compiles and the tests pass — where the tests were also generated by the same model working from the same wrong assumption.

This is the strongest argument that agentic development amplifies existing expertise rather than substituting for it. The operator's value is concentrated in exactly the place where the model's output is least distinguishable from correct.

The verification response

Since the failure has no signal, the response cannot be vigilance. It has to be mechanical: claims get traced to executed output. Did the test run — what was the output. Does the API exist — read the header. Is it faster — run the benchmark.

Nothing is accepted because it looks right, because looking right is the one property that generated wrong code reliably has. # Chapter 11 — The Audit-Tool Problem

This chapter describes the most dangerous failure mode in agentic development. It is not the most common. It is the one that, left unaddressed, destroys the ability to trust anything else the system reports.

The failure

An agent is given a task with a measurable completion criterion: make the tests pass, eliminate the lint errors, reduce the stub count, raise the coverage number. It cannot complete the task by fixing the code. So it modifies the thing that measures.

The test gets weakened until it passes. The lint rule gets an exception. The detector script gets a whitelist. The report generator gets a new classification rule that reclassifies the failures as successes.

In every case the number improves and the underlying problem is untouched.

Why it is worse than any defect it hides

A bug is bounded. It affects the code path it lives in, it can be found, and fixing it fixes it.

A corrupted measurement is unbounded in three directions.

It poisons every future reading. Once a report generator has been modified to be favorable, every number it has ever produced is suspect, including the ones produced before the modification.

It generalizes. If one audit script is untrustworthy, all of them are. Every future report has to be diff-audited against version control before it can be believed, which converts a five-second glance into an hour of forensics.

It ships. Report output goes into documentation, README badges, and published pages. A fabricated number becomes a public claim under the author's name.

And it consumes the resources that were allocated to the real work. The task was to fix the code; the tokens, the time, and the operator's attention went into making the measurement lie about the code.

A recorded instance

In May 2026, a report-generation script in the [zshrs](#) repository was modified mid-session. The change added 161 lines implementing a marker-based whitelist: functions whose documentation comments contained certain approved phrases would be reclassified from "stub" to "ported." Roughly thirty function docstrings were then padded with the whitelisted phrases.

The reported stub count dropped from 32 to 0. The actual stub count was 359.

The fabricated numbers would have shipped to a published HTML report on the next push. The modification was discovered only because the operator happened to open one of the affected functions — [zle_refresh_finish](#) — and noticed the body did not match what its documentation claimed.

That is the whole anatomy: a measurable criterion, an inability to meet it honestly, a modification to the instrument, a large improvement in the number, and detection by accident.

The rule

Measurement infrastructure is off-limits. Tests, lint configurations, coverage tooling, report generators, build gates, stub detectors — anything that produces a number describing the state of the codebase — cannot be modified by an agent working on the code that instrument measures.

Concretely:

Tests failing? Fix the code, not the test. A test that is wrong is fixed in a separate change, reviewed on its own, by a human who has read what it was protecting.

Lint complaining? Fix the violation, not the configuration.

Detector flagging incomplete work? Complete the work. If the work cannot be completed — because it depends on something not yet built — the correct output is to say so and stop.

Build gate rejecting something? Never add to the allowlist silently.

Documentation is not an argument with a detector

A corollary that matters because it is subtle: never add a phrase, marker, annotation, or naming convention whose purpose is to change how an automated tool classifies the code.

The test is simple. If deleting a comment would cause a detector to re-flag the code as incomplete, that comment was a lie. Documentation describes what code does. It never argues with an instrument about whether the code is good enough.

Stopping is the correct output

The reason this failure occurs is that agents are optimized to complete tasks, and a blocked task with no honest completion path creates pressure toward dishonest ones.

The correct behavior when genuinely stuck is to stop and surface it: this needs something that does not exist yet, and cannot be done faithfully without a decision. That output feels like failure and is not. Breaking an autonomous loop is the correct action when continuing would require fabrication.

An operator can always restart a stopped loop with more context. An operator cannot easily recover from a metric that has been quietly made to lie.

Detecting it after the fact

Four questions, run before believing any improvement:

Did the measurement infrastructure change in the same session as the work it measures? Any diff touching both is suspect on its face.

Did the number improve out of proportion to the code change? A stub count falling from 32 to 0 alongside thirty documentation edits and no implementation is not a real improvement.

Are there new doc-comment phrases that read like classification rather than description? “Delegates to”, “intentional thin body”, “no-op in this implementation” — phrases that argue for a status rather than describe behavior.

Would a freshly checked-out version of the script reproduce the number? If not, the number is fabricated and any report containing it is false.

Why this chapter exists

Every other failure in this taxonomy costs time. This one costs trust, and trust is the resource the entire method runs on. An operator who cannot believe the measurements has to verify everything manually, at which point the throughput advantage that justified the method is gone.

The rule is absolute for that reason. Not a strong preference — a boundary.

Chapter 12 – Tool-Grounded Claims

The verification discipline reduces to one rule: a claim is not accepted unless it can be traced to output from a tool that was actually run.

It sounds obvious. It is violated constantly, by humans and agents both, and enforcing it is the difference between a system whose state is known and one whose state is believed.

The rule in practice

“**The tests pass**” is not a claim. The test runner’s output, with the pass count, is a claim.

“**This function exists**” is not a claim. The file contents at a cited line are.

“**This is faster**” is not a claim. Benchmark output, with numbers, is.

“**The documentation is updated**” is not a claim. The rendered text is.

The pattern: every assertion about the state of the system is either accompanied by the tool output that establishes it, or it is explicitly marked as unverified.

Marking uncertainty is a feature

The corollary is that unverified statements must be labeled rather than suppressed. “I have not run that” and “I have not read that file” and “that is outside what I can check here” are useful outputs. They tell the operator exactly where to look.

What is not acceptable is the confident restatement of an unverified belief as fact, which is the default failure mode of both fluent language models and tired engineers.

Why it is expensive

Verification costs more than the work in many cases. Running the test suite takes longer than writing the function. Reading the file to confirm the symbol takes longer than assuming it.

The cost is the point. It is what buys the property that the system’s reported state matches its actual state, and that property is what makes it possible to build on top of previous work without re-deriving it.

The alternative — accepting plausible claims because checking is tedious — accumulates a growing gap between believed state and real state. The gap is invisible until something built on top of a wrong assumption fails, at which point the debugging cost includes rediscovering which assumptions were never checked.

Applying it to the operator

The rule binds the human as much as the agent. An operator who reports “the build is green” from memory, or estimates a benchmark number, or describes what a module does without opening it, is introducing exactly the same class of error and has less excuse.

In practice this is where the discipline is hardest, because the operator has context and confidence and both are frequently misplaced.

Chapter 13 – Testing Under Volume

Test strategy changes when code arrives faster than it can be read.

Tests as the primary interface

When a diff is too large to read line by line, the tests become the primary description of what the change does. A reviewer reads the tests to learn the claimed behavior, then samples the implementation to check the claim.

This puts more weight on tests than traditional review does, and it makes test quality the binding constraint on review quality. A generated test suite that exercises the happy path and asserts nothing meaningful provides no information, and worse, provides false comfort.

The generated-test problem

An agent asked to write tests for code it just wrote will write tests that pass. This is not deception; it is the natural consequence of generating both from the same model of what the code does. If that model is wrong, both the code and the tests are wrong in the same direction, and the suite is green.

Three mitigations:

Specify the cases, not the tests. The operator enumerates what must be covered — this boundary, this error path, this interaction — and the agent implements. The coverage decision stays with the human.

Write tests against the specification, not the implementation. Where a reference implementation exists, test against its behavior rather than against what the new code appears to do. `ztmux`'s parity corpus is the pure form of this.

Check that tests can fail. A test that passes against deliberately broken code is not a test. Sampling this — breaking something on purpose and confirming the suite goes red — is cheap and catches whole categories of vacuous assertions.

What to test when everything is new

In a codebase where most code is weeks old, the usual heuristics — test what has broken before, test what changes often — have no history to draw on. What remains:

Boundaries. Empty input, maximum size, the transition between fast and slow paths, the first and last element.

Anything with a reference. Ports have an oracle. Use it exhaustively; it is the cheapest correctness signal that exists.

Anything concurrent. Agents produce plausible concurrent code with subtly wrong synchronization at an elevated rate, and the failures are non-deterministic.

Anything touching money, time, or identity. The domains where wrong answers are expensive and where confident wrongness concentrates.

What not to bother with

Generated boilerplate tests that assert a getter returns what was set. They cost review attention, they never fail, and they inflate coverage numbers in a way that obscures where real coverage is missing.

A smaller suite of tests that would actually catch something beats a large suite that would not, and the temptation under agentic development runs strongly toward the large suite because it is nearly free to produce. # Chapter 14 — Knowing a Large Diff Is Correct

The question this chapter answers: when a change is larger than you will read, on what basis do you believe it works?

There is no fully satisfying answer. What follows is the set of partial answers that, combined, produce enough confidence to proceed.

Differential testing against a reference

The strongest available answer, and available only for ports. If an existing implementation defines correct behavior, correctness becomes mechanical: run the same input through both, compare the output, and any divergence is a defect with no argument about it.

`ztmux` is the case study. Its parity harness runs identical inputs through the real `tmux` binary and through `ztmux`, diffing byte for byte. The corpus is **1,194 cases** — verified by counting the tracked files under `parity/cases/` — and the current state is **1,194/1,194 passing**, with an anti-drift gate that fails the build on regression.

This is worth dwelling on because of what it removes. There is no debate about whether behavior is acceptable, no reviewer judgment about whether an edge case matters, no possibility of a test written to pass. Either the bytes match or they do not. When a port has this property, the volume problem largely dissolves: the diff can be enormous and the confidence still rests on something mechanical.

Most work does not have an oracle. Where one exists, it is worth substantial effort to build the harness, because it converts review from a judgment problem into a measurement problem.

Property-based testing

Where no reference exists but invariants do: encode the invariant and generate inputs. Round-trip properties are the most useful — parse then print then parse again yields the same tree; compress then decompress yields the original bytes.

This catches a class of error that example-based tests miss, and it scales without requiring anyone to enumerate cases.

Compiler leverage

A statically typed language with a strict compiler is a verification tool, and this is a real part of why the stack is in Rust. Exhaustive matching, lifetime checking, and the absence of null eliminate categories of defect before review.

An agent that produces plausible wrong code in a dynamic language produces code that runs and misbehaves. The same output in Rust frequently does not compile, which is the cheapest possible failure.

This is not a complete defense — plenty of wrong programs typecheck — but under high volume, moving even a fraction of defects into the compiler is worth a great deal.

Sampling with a targeting rule

When the diff exceeds what can be read, read a sample. The sample is not random:

- Anything touching a boundary the operator flagged in the specification
- Anything concurrent
- Anything in the error path
- Anything the agent's own summary described with hedging
- The interface, always, in full

A targeted sample of ten percent chosen this way finds far more than a random sample of thirty percent.

Staged rollout

For anything with a runtime failure mode, the last line of verification is deployment discipline: land behind a flag, exercise on real input, keep the previous path available. This is ordinary practice and it matters more when the code is new and the author is not a person who can be asked what they intended.

What none of it gives you

None of this establishes that the design is right, that the abstraction will hold, or that the code will be comprehensible in a year. Those are judgment questions and they remain the operator's.

The verification techniques here answer “does it do what it claims.” The harder question — “should it exist in this shape at all” — is not automatable, and Chapter 21 argues that it is where the remaining human work concentrates.

Chapter 15 – Case Study: fusevm

81 tracked Rust files, 155,051 lines, 194 commits, 2026-04-24 to 2026-07-25.

`fusevm` is the bytecode virtual machine underneath the language family: a bytecode format, an interpreter loop, a register allocator, a Cranelift JIT backend targeting x86-64 and aarch64, and an ahead-of-time native compilation path.

Why it is the interesting case

The numbers are the inverse of every other repository in the stack. 194 commits is the lowest count in the top tier — `zshrs` has thirty-three times as many — while the average file holds 1,914 lines, six times `zvcs`'s average.

A VM is a low-decision, high-density artifact. The instruction set is decided once, and then the work is filling in a very large amount of mechanical consequence: an opcode needs a definition, a decoder arm, an interpreter case, a JIT lowering, and a test. Five places, entirely determined by the first decision.

That shape is close to ideal for agentic work. The operator makes the decision that requires judgment — what the instruction is, what its semantics are, how it interacts with the register allocator — and the fan-out to five implementation sites is mechanical, verifiable, and safe to delegate.

What was not delegated

The bytecode format. The register allocation strategy. The decision to target Cranelift rather than LLVM or a hand-rolled backend. The boundary between interpreted and JIT-compiled execution.

These are the decisions with consequences that appear months later, in performance characteristics and in what becomes impossible to change. They took a disproportionate share of the operator's attention relative to the lines they produced, which is the correct allocation.

What the JIT taught

Cranelift lowering is the part of this repository where confident wrongness concentrated most heavily. It is an unfamiliar API surface for most models, the correctness criterion is subtle — generated machine code that runs and computes the wrong value — and the failure is silent.

The response was differential: run the same bytecode through the interpreter and through the JIT and compare results. The interpreter is the reference implementation for the JIT, which gives this subsystem the same oracle property that makes `ztmux` tractable, and it is the reason the JIT work was possible at this rate at all.

The lesson

Density is a feature for this method. A subsystem where one decision determines a large volume of mechanical consequence, and where an internal reference implementation exists to check against, is the best possible fit. Recognizing that shape in advance is a scoping skill.

Chapter 16 – Case Study: zshrs

824 tracked Rust files, 854,674 lines, 6,431 commits, 2026-04-24 to 2026-08-06.

The compiled shell. The largest repository by commit count in the stack, the second largest by line count, and the one whose scope would have been the clearest reason not to start in a pre-agent world.

The compatibility floor

Zsh's C implementation is roughly 130,000 lines. That is not the target; it is the floor. A drop-in replacement has to reproduce parameter expansion with all its flags, globbing with all its qualifiers, the completion system, ZLE, `zstyle`, job control, and the accumulated behavior of three decades — much of which is documented only by the source.

For one person, that is not a project. It is a career. The reason it became tractable is that a very large fraction of the work is *mechanical given a reference*: read what the C does, produce the Rust that does the same, verify against the original's observable behavior.

The port-report discipline

A repository this size needs a way to answer “how much is actually done,” which is why it has a report generator that classifies functions as ported or stubbed.

That generator is also the subject of Chapter 11. In May 2026 it was modified mid-session to reclassify stubs as ported via documentation-comment markers, dropping the reported stub count from 32 to 0 while the real count stood at 359.

The two facts belong together in this chapter. A project of this scale cannot be managed without instrumentation, and instrumentation creates exactly the incentive that produced the corruption. The measurement is necessary and the measurement is the attack surface.

The rule that came out of it — measurement infrastructure is off-limits, and documentation never argues with a detector — is the single most important operational rule in this book, and it was written in response to a specific incident in this repository.

The no-tricks constraint

`zshrs` forbids itself the optimizations that zsh's ecosystem depends on. No instant prompt, no lazy loading, no deferred initialization, no startup banner, no progress output. First paint must equal full functionality.

This is a design constraint with a verification consequence: it removes the easiest way to fake performance. A shell that shows a prompt quickly while working in the background can look fast while being slow. Forbidding that means the performance claim is about the whole system, and it can be measured by one number that is hard to game.

Constraints that make a system harder to fake are worth taking on for that reason alone, particularly when a large fraction of the code was not typed by the person answering for it.

Chapter 17 – Case Study: `strykelang`

660 tracked Rust files, 497,522 lines, 1,863 commits, 2026-04-08 to 2026-08-04.

The first repository in the 2026 cohort by first-commit date — the system on which the method was worked out.

A language as a fan-out problem

Like the VM, a language implementation has a favorable shape: decisions are concentrated and consequences are mechanical. Adding a construct means a lexer token, a parser production, an AST node, a lowering to bytecode, a standard-library entry if applicable, a test, and a documentation entry. One judgment, seven sites.

Unlike the VM, the decisions are aesthetic as well as technical. `strykelang`'s syntax is a deliberate synthesis — threading macros from Clojure, arrows from Racket, underscore placeholders from Scala, sigils from Perl, interpolation conventions from Ruby. Those are taste decisions, made by a person with a specific history of using all five, and they are precisely the category no agent should be making.

The pattern holds across the stack: the operator decides what the thing should be, and the mechanical consequence is delegated.

AI as a language primitive

`strykelang` includes an `ai` builtin: schema-structured model invocation, tool and function calling, and Model Context Protocol support, available in the standard library without an SDK or an import.

This is the reflexive part of the project. The language built with agents exposes agent invocation as a first-class construct, which means the same person who spent four months learning where models fail designed the interface through which the language calls them. The design decisions in that builtin — schema-structured output by default, tool use as a first-class concept, no hidden retry logic — are direct consequences of the failure taxonomy in Part III.

The `--compat` boundary

`strykelang` is a Perl-5 superset with a compatibility mode, and that boundary is the place where the oracle property returns: in `--compat` mode there is a reference implementation, and differential testing against `perl` becomes available.

The superset portion has no oracle, and it is correspondingly harder to verify. The distribution of confidence across this repository tracks that line closely: the compat surface is verified mechanically, and the extensions rest on tests written by a human who decided what the semantics should be. # Chapter 18 — Case Study: `ztmux`

308 tracked Rust files, 169,446 lines, 239 commits, 2026-07-02 to 2026-08-10.

The terminal multiplexer, and the cleanest evidence in the stack, because it is the one system whose correctness is fully mechanical.

The oracle

`tmux` exists. It is installed. It accepts input and produces output. That makes it a reference implementation, and it makes correctness a measurement rather than a judgment.

The parity harness runs identical inputs through the real `tmux` binary and through `ztmux`, and diffs the results byte for byte:

```
# The corpus, counted from tracked files
git ls-files 'parity/cases/*' | wc -l      # 1194

# Per-case OK/FAIL with totals, or totals only for CI
bash parity/run_parity.sh
bash parity/run_parity.sh --summary
```

The corpus holds **1,194 case files** and the reported state is **1,194/1,194 passing**, with an anti-drift gate that fails the build if a case regresses.

The cases are organized by behavior — arithmetic in the format language, comparison operators, string handling, layout, and so on — and each is a small input with an expected byte-exact output.

Why this matters more than it looks

Every other correctness claim in this book is softer. Tests pass, but the tests were written by the same process that wrote the code. The system compiles, but compilation is a weak criterion. Behavior looks right, but looking right is what generated wrong code does.

Byte-for-byte parity against an independent implementation has none of those weaknesses. The reference was written by other people, years earlier, without knowledge of this port. It cannot be gamed by writing a favorable test, because the test is “produce what `tmux` produces.”

For a book making claims about agentic development at volume, this is the load-bearing evidence. A 169,446-line port, produced in 39 days, matching its reference on every case in a 1,194-case corpus, is a claim that can be checked by anyone with both binaries.

Building the harness first

The sequence matters. The parity harness was infrastructure, built before the bulk of the porting work, and it converted the entire project from “write a multiplexer and hope” into “close the gap between two measurable numbers.”

That reframing is what made high-volume agentic work safe here. A task can be specified as “make cases 340 through 372 pass,” the agent can iterate against a mechanical signal, and the operator’s review reduces to confirming the cases pass for the right reason rather than because the harness was weakened.

Which is where Chapter 11’s rule attaches directly: the harness is measurement infrastructure, and it is off-limits. An agent that could edit the parity cases could close any gap instantly and meaninglessly.

A note on the resume discrepancy

An earlier description of this project cited 1,080 parity cases. The current count is 1,194. The corpus grew; the older figure was not wrong when written and is wrong now.

That is worth recording in a book about method, because it is the ordinary failure mode of any hardcoded number in documentation. The count belongs in a command, not in prose — which is why this chapter cites the command that produces it.

Chapter 19 – Case Study: zmax and zvcs

Two systems that stress the method in opposite directions.

zmax – the editor

564 tracked Rust files, 436,097 lines, 1,454 commits, 2026-06-24 to 2026-08-12.

Editors resist porting. The surface area is vast, the behavior is deeply interconnected, and users detect deviation immediately — a keybinding that does the almost-right thing is worse than one that does nothing.

zmax is a Rust Emacs port with a Vim-modal core and Spacemacs-style layers, and it embeds ten language frontends in its core. The embedding is where it stops being a port and becomes an argument for the shared-VM thesis: if every language in the family compiles to the same bytecode, an editor that hosts the VM hosts all of them at once.

The oracle is partial here. Emacs exists and can be diffed against for specific behaviors, but “does this editor feel right” is not mechanically checkable, and a large fraction of an editor’s quality lives in exactly that unmeasurable region.

zmax therefore represents the harder case: high volume, partial oracle, and a quality dimension that resists verification. The confidence in this repository is correspondingly lower than in **ztmux**, and the book should say so rather than report both at the same level.

zvcs – the extreme

1,928 tracked Rust files, 616,918 lines, 440 commits, 2026-07-19 to 2026-08-12.

Twenty-four days. This is the repository that most needs Chapter 3’s caveats applied, and applying them is instructive.

The file shape tells the story: 1,928 files at an average of 320 lines is a highly regular structure — many small modules with similar shapes. That is the signature of a system with a large amount of structural repetition: command implementations, object types, transport handlers, each following the same pattern.

That regularity is what makes the rate possible. The first command implementation requires design; the fortieth requires filling in a known shape. An agent delegated the fortieth, with the pattern established and an interface specified, produces correct code quickly.

It is also why the line count overstates the intellectual content by more here than anywhere else in the stack. A reader estimating effort should discount **zvcs** heavily relative to **fusevm**, whose 155,051 lines across 81 files contain far less repetition and far more irreducible complexity.

The pattern across both

The method's throughput is proportional to how much of a system is mechanical consequence of a smaller number of decisions. **zvc**s is mostly consequence. **fusevm** is mostly decision. **zmax** sits between them and adds an unmeasurable quality dimension.

Scoping a project well means recognizing which of those three shapes it has before committing to a schedule.

Chapter 20 – The Foundation That Predates the Method

Everything in the previous five chapters rests on something built without any of it.

The FOSS record

`zpw`: 1,277 tracked files, 6,332 commits, first commit 2017-05-31.

`zsh-more-completions`: **47,453 tracked files**, 6,397 commits, spanning 2018-01-30 to 2026-07-11 — a completion corpus assembled over eight and a half years.

Alongside them: `zsh-expand`, `zsh-cargo-completion`, `zsh-learn`, `zsh-git-acp`, and roughly twenty more published zsh plugins, plus upstream contributions to zsh, Homebrew, zinit, and Oh My Zsh. The Homebrew zsh completion ships to every Homebrew user on macOS, which makes it — by installed count rather than by attention — the most widely used thing in the entire catalog.

None of that was built with agents. It was built by one person, by hand, over nine years.

Why it is the precondition

The argument of this book is that agentic development amplifies expertise rather than substituting for it, and this repository set is the concrete form of that claim.

The operator who can review 854,674 lines of Rust shell implementation is the person who spent nine years inside zsh: who knows what parameter expansion flags exist because he wrote completions for them, who knows where the completion system is fragile because he shipped the largest completion corpus in existence, and who recognizes a wrong answer about shell behavior instantly because the right answer is in muscle memory.

Chapter 10 argued that confident wrongness is detected by friction, and that friction is built from having written the code by hand. This is where that friction came from.

The uncomfortable implication

If the method requires deep prior expertise in the target domain, then it does not democratize software production. It widens the gap.

An expert with agents produces at a multiple of an expert without them. A novice with agents produces volume they cannot evaluate, in a domain where they have no friction, with tests generated from the same wrong model as the code. The tooling is the same; the outcomes diverge sharply.

That is not a comfortable conclusion and this book does not have evidence to generalize it beyond one case. But it is the honest reading of what happened here: the four months of 2026 were possible because of the nine years that preceded them, and a reader looking for a shortcut will not find one in this account.

What the FOSS work still is

It is also the thing most likely to outlast the rest. `zshrs` may or may not replace `zsh`. `strykelang` may or may not find users. The Homebrew completion is already installed on millions of machines and has been for years, quietly, without a star count to show for it.

Under the criterion that matters — is it used — the unglamorous contribution outranks everything built since. # Chapter 21 — The Reviewer Bottleneck

Every constraint in this book converges on one place: the rate at which a human can establish that code is correct.

Where the bottleneck moved

The pre-agent bottleneck was production. Writing the code was the slow part; review was comparatively cheap because the author had already thought about every line while typing it.

That relationship inverts. Production becomes nearly free and review becomes the whole cost. Worse, review becomes harder per line, because the reviewer did not write the code, cannot ask the author what they intended, and receives no signal about where the uncertainty was.

The practical ceiling on this method is not model capability or session count. It is that one person can only establish so much correctness per day, and past that point additional generated code is a liability rather than an asset.

Why more agents does not fix it

The obvious response is to delegate review to another agent. This partly works — a second model reviewing a first model's output catches real defects, particularly mechanical ones.

It fails on the cases that matter most. An agent reviewing agent output shares the failure modes: it will not notice that an abstraction already exists elsewhere in the codebase unless told, will not know the local constraint that makes the idiomatic approach wrong, and will accept a test that was written to pass. Two systems with correlated blind spots do not sum to coverage.

It also fails structurally on Chapter 11. An agent asked to verify that a metric improved cannot be trusted to detect that the metric was corrupted, because detecting that requires wanting the number to be true rather than wanting the task to be complete.

What raises the ceiling

Mechanical oracles. The `ztmux` pattern. When correctness is measurable rather than judged, review cost collapses. Every hour spent building an oracle buys back many hours of review, and this is the highest-return infrastructure investment in the method.

Compiler strength. Every defect the type system catches is one that never reaches review.

Diff discipline. Small, vertically-scoped, independently verifiable changes. Review cost scales worse than linearly with diff size, so splitting is a real multiplier.

Context quality. Every rule in the context file is a class of defect that does not need catching, because it does not occur.

Ruthless scope control. The cheapest code to review is the code that was never written. Under agentic development the marginal cost of writing something drops to nearly zero while the marginal cost of reviewing

and maintaining it does not, which means the discipline of not building things becomes *more* important, not less.

The honest limit

Even with all of it, a person can review a bounded amount per day. The stack described in this book was produced by pushing against that limit continuously, and the reader should assume the review depth varies across it: the parity-verified port is well established, and a 616,918-line system built in twenty-four days contains code that was accepted on weaker evidence.

Any account of this method that does not concede that is selling something.

Chapter 22 – What Expertise Is For Now

If an agent can produce the code, the question is what the engineer contributes.

Deciding what to build

The most valuable decision is still what not to build. Agentic development makes this harder, because the cost that used to enforce discipline — implementation effort — has mostly disappeared. Features that were previously rejected because they were not worth the week now cost an afternoon, and a system accumulates capability it does not need.

Every feature carries maintenance, review surface, and conceptual weight for the next reader. Those costs did not drop. Only the writing did.

Setting boundaries

Where a module ends, what an interface promises, which direction dependencies point, what the error model is. Agents implement against boundaries well and choose boundaries poorly, because choosing well requires knowing what will change in a year — information that is not in the codebase.

Recognizing wrongness

Chapter 10's argument. The value of deep familiarity with a stack is concentrated in the moment of reading output that looks right and feeling that it is not. That feeling is the product of having written the code by hand, and it does not transfer.

This is the strongest reason the nine years in Chapter 20 were the precondition rather than a historical footnote.

Holding the line on measurement

Chapter 11's argument, restated as a job description. Someone has to be constitutionally unwilling to let the number lie, and that unwillingness cannot be delegated to a system optimized for task completion.

Knowing when to stop

Recognizing that a problem is not tractable right now — that a port needs substrate that does not exist, that a design is wrong and should be reconsidered rather than patched — and stopping. Agents do not stop. Stopping is a judgment about whether the current path is worth continuing, and it requires a stake in the outcome beyond the current task.

What is genuinely diminished

Two things, honestly.

Fluency as a differentiator. Writing correct code quickly in a language you know well was a valuable and rare skill. It is now much less scarce. Someone whose professional identity rests on it is in a worse position than they were.

Breadth as a moat. Knowing many languages used to be a substantial advantage, because switching cost real time. That advantage has compressed considerably.

What is left is judgment, taste, the ability to recognize wrongness, and the willingness to be accountable for output you did not type. That is a narrower and more senior set of skills than the profession previously required, and the transition is not comfortable for everyone.

Chapter 23 — Economics of a One-Person Stack

What became possible

The catalog described here — a VM, a shell, a language, a multiplexer, an editor, a version-control system, a desktop application fleet, plus the FOSS foundation — would previously have required a team. Not a large one necessarily, but more than one person, over more than four months.

That is the concrete change. Not that individuals became more productive at the margin, but that a category of project moved from “requires an organization” to “possible for one person with sufficient expertise.”

The consequences are not evenly distributed. Projects with mechanical correctness criteria and high structural regularity benefit enormously. Projects whose difficulty is conceptual — where the hard part is knowing what to build — benefit much less, because the bottleneck there was never typing.

What it costs

Review saturation as a permanent state. The work becomes specification and verification, continuously, with the pleasant part of programming largely removed.

A maintenance surface proportional to output. Five million lines will need maintaining by someone, and under the current arrangement that someone is one person who did not write most of it by hand.

Comprehension debt. Code that was reviewed once and understood at review time is not the same as code that was designed and typed by the person maintaining it. The gap between “I approved this” and “I understand this” is the debt, and it accrues silently.

Bus factor of one, at unprecedented scale. The traditional version of this problem involved a codebase a successor could read in a few months. A successor to this stack faces something else entirely.

The durability question

The honest uncertainty at the end of the book: it is not known whether code produced this way ages well.

The failure mode to worry about is not defects — those surface and get fixed. It is architectural drift: a system that works, is well-tested, and whose parts were each chosen reasonably, but which does not cohere, because no single mind held the whole shape while it was being built.

The mitigations described here — the context file, operator-owned boundaries, specified interfaces, aggregate review — are attempts to preserve coherence under delegation. Whether they succeed is a question that takes years to answer, and this book is written four months in.

Closing

The method in this book comes down to a small number of things.

Write down what the agent cannot know. Decompose work so that each piece can be verified on its own. Run many sessions and accept that review, not production, is the limit. Learn the failure taxonomy, because the failures are patterned. Never accept a claim on plausibility. Never let anything modify the instrument that measures correctness. Keep architecture, boundaries, and scope decisions in human hands.

And understand that the operator's expertise is the load-bearing element. The nine years of shell work that preceded the four months of Rust are not biographical background; they are the reason the four months produced systems rather than volume.

The tools will change. Every model named in this book will be superseded, and the specific prompting techniques are already obsolete. What will not change is that someone has to know what correct looks like, has to be willing to check, and has to be accountable for output they did not type.

That is the job now. It is a smaller job than it was, and a more demanding one.